TECHCATALYST DE 2026

Data engineering foundations

What data engineers do, the data they work with, and the modern data stack you will master this summer.

Week 1
Day 1

- What is data engineering
- Types of data & formats
- Batch vs streaming
- The modern data stack
- Analytics & team roles

Your 8-week journey

Week	Theme	Week	Theme
1	Data & cloud foundations	5	Big data & PySpark
2	Developer foundations — Linux, Git & Python	6	GenAI & AI-assisted engineering
3	Warehousing & SQL on GCP	7	BI, visualization & data apps
4	Snowflake, dbt & advanced SQL	8	Capstone

One continuous project thread: you will design, build, and ship a large-scale NYC Taxi data pipeline — starting with today's concepts and ending with your capstone demo.

DAY 1

Today's agenda

Time	Block
8:00 – 9:00	Welcome, introductions, program expectations
9:00 – 10:30	What is data engineering? Roles and the DE lifecycle
10:45 – 12:00	Types of data, file formats, batch vs streaming
1:00 – 2:30	The modern data stack & The Hartford's stack
2:45 – 3:45	Group activity 1 — Analytics in action
3:45 – 4:45	Group activity 2 — DE case study: car insurance
4:45 – 5:00	Readouts, recap, and what's next

PRACTITIONER FIRST

Strategy, systems, and teaching grounded in real delivery.

CONSULTANT / EDUCATOR / AUTHOR

TAREK ATWAN

AI & Data Advisor · Educator · Author

I help organizations turn AI ambition into production capability — and help professionals build the judgment to lead it.

WORKING ACROSS

Generative AI · Machine Learning · Data Engineering
Cloud · Cybersecurity

20+	85+	47+	4
YEARS	COURSES	ENTERPRISE CLIENTS	BOOKS

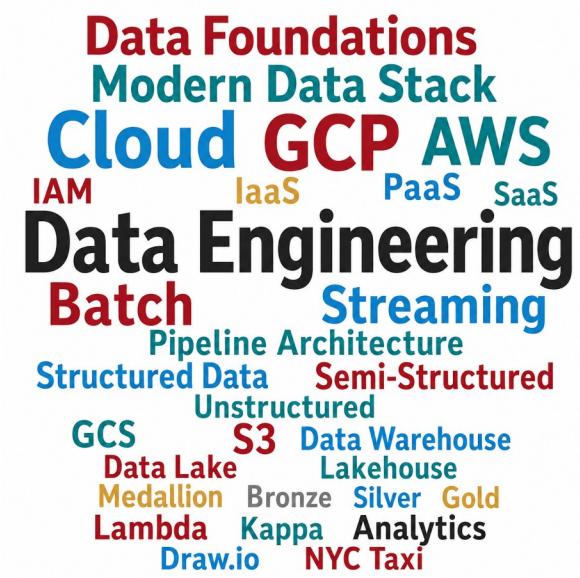
tarekatwan.com

AMMAN · US · MIDDLE EAST · EUROPE

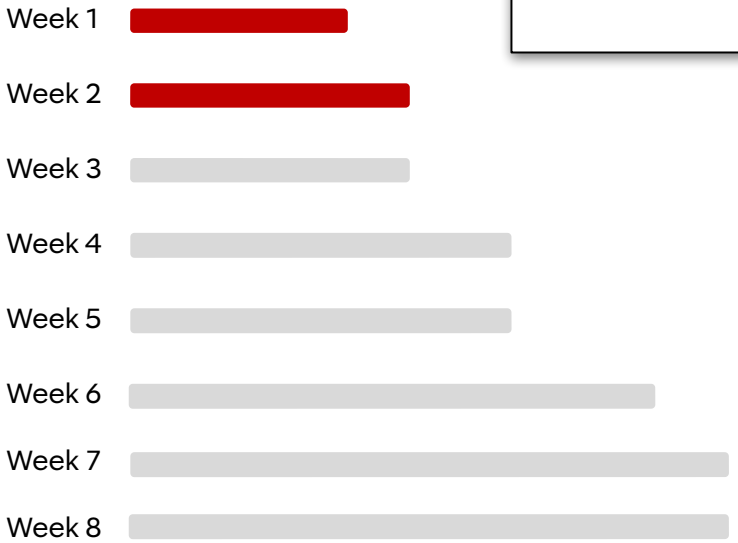
Your 8-week journey - Week 1

Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Week 7
Week 8

Data Engineering and **Cloud Foundations**: Introduction to Data Engineering Fundamentals including Data Architecture Fundamentals, Data Engineering Primer, and Data Engineering on the cloud (GCP and AWS).

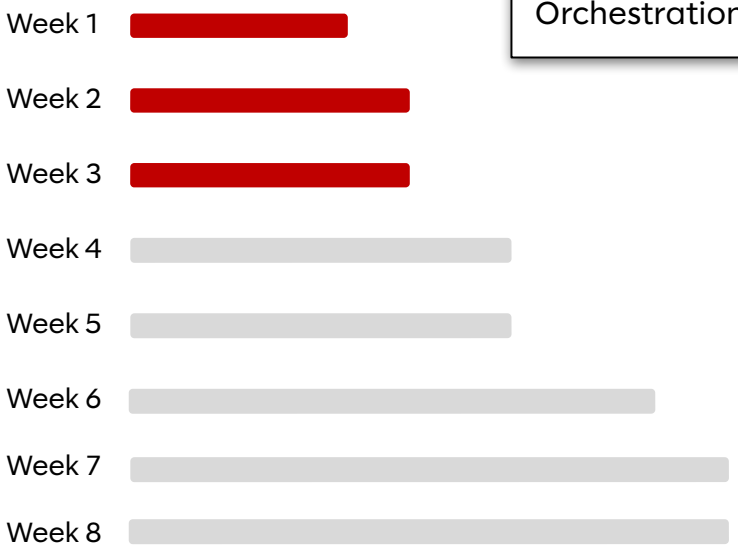


Your 8-week journey - Week 2



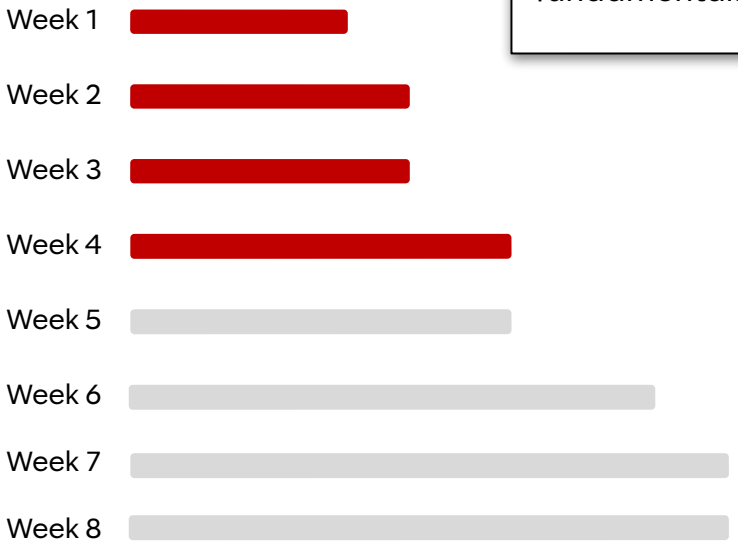
Developer Foundations: Python fundamentals, Linux CLI & Git, Pandas * Polars for Data Engineering.

Your 8-week journey - Week 3

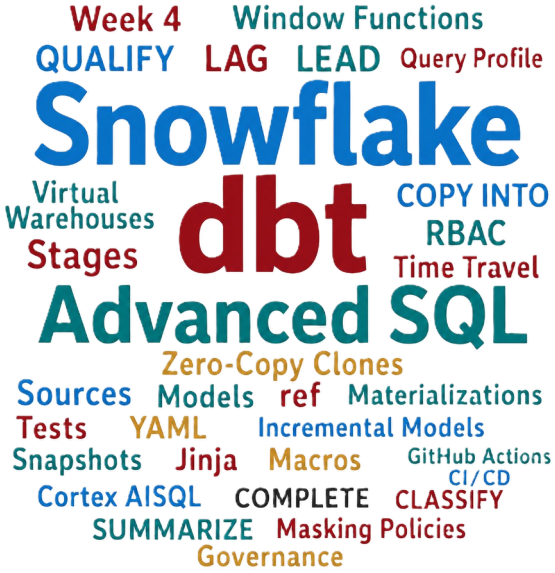


Modern Data Warehousing & SQL: Data Lakes, Lakehouses, ETL vs ELT, Snowflake vs Star Schema, Google BigQuery, SQL Primer, Orchestration.

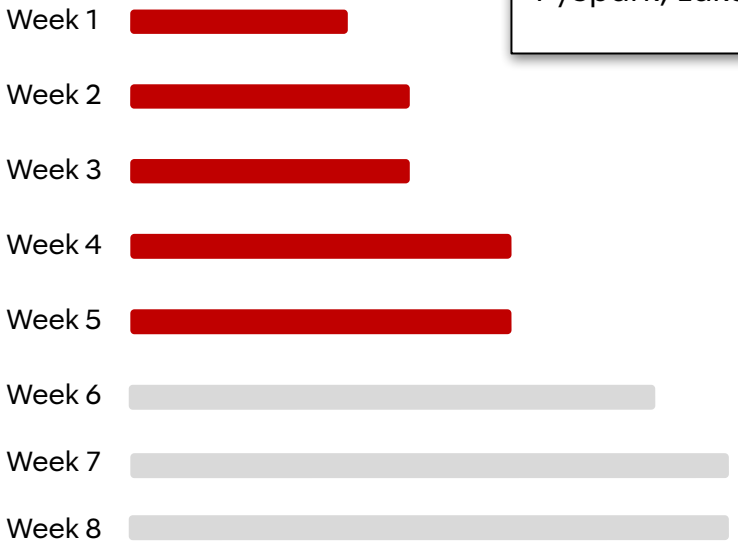
Your 8-week journey - Week 4



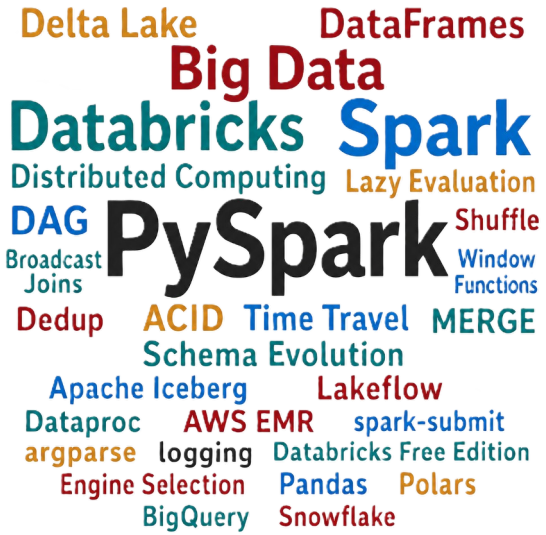
Snowflake Database: Snowflake architecture, advanced SQL, dbt fundamentals, Snowflake GenAI, Advanced dbt



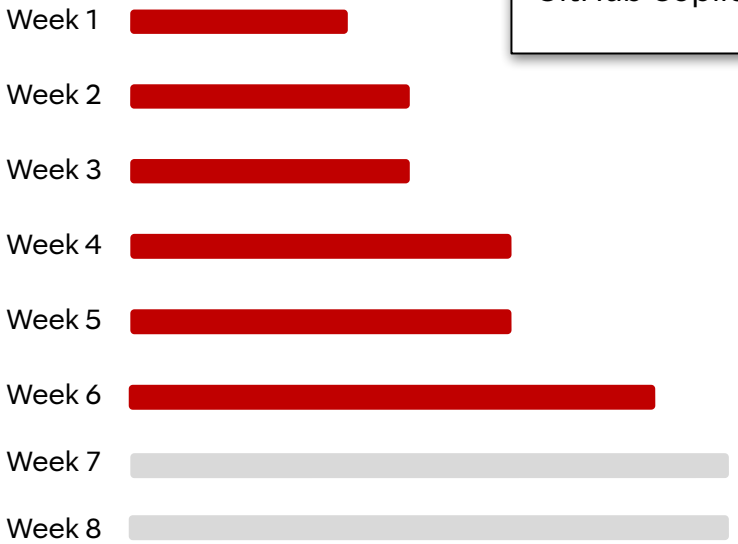
Your 8-week journey - Week 5



Big Data & PySpark ETL: Big Data, Spark Foundations, Databricks & PySpark, Lakehouse formats (Delta Lake, Hudi, and Iceberg).

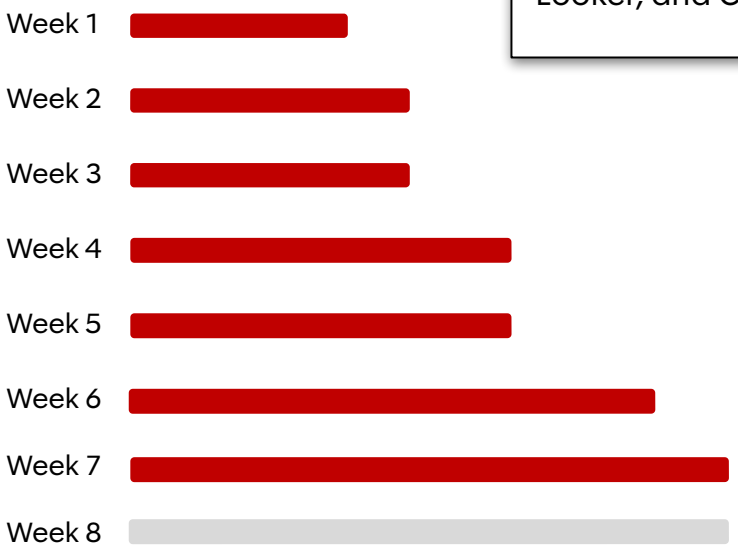


Your 8-week journey - Week 6



NLP, GenAI , and LLMs: GenAI Foundations, Python API integration, GitHub Copilot, Vertex AI, Applied NLP, Coding Agents.

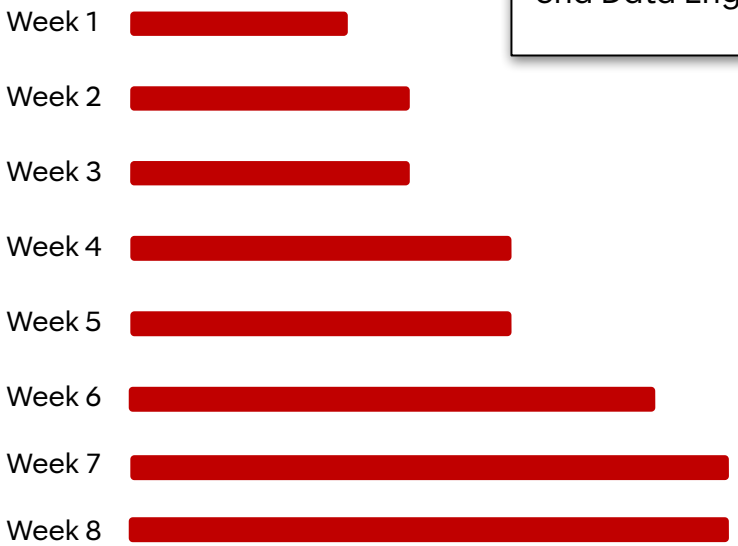
Your 8-week journey - Week 7



Data visualization: Storytelling, Streamlit, Tableau, ThoughtSpot, Looker, and Capstone preparation.

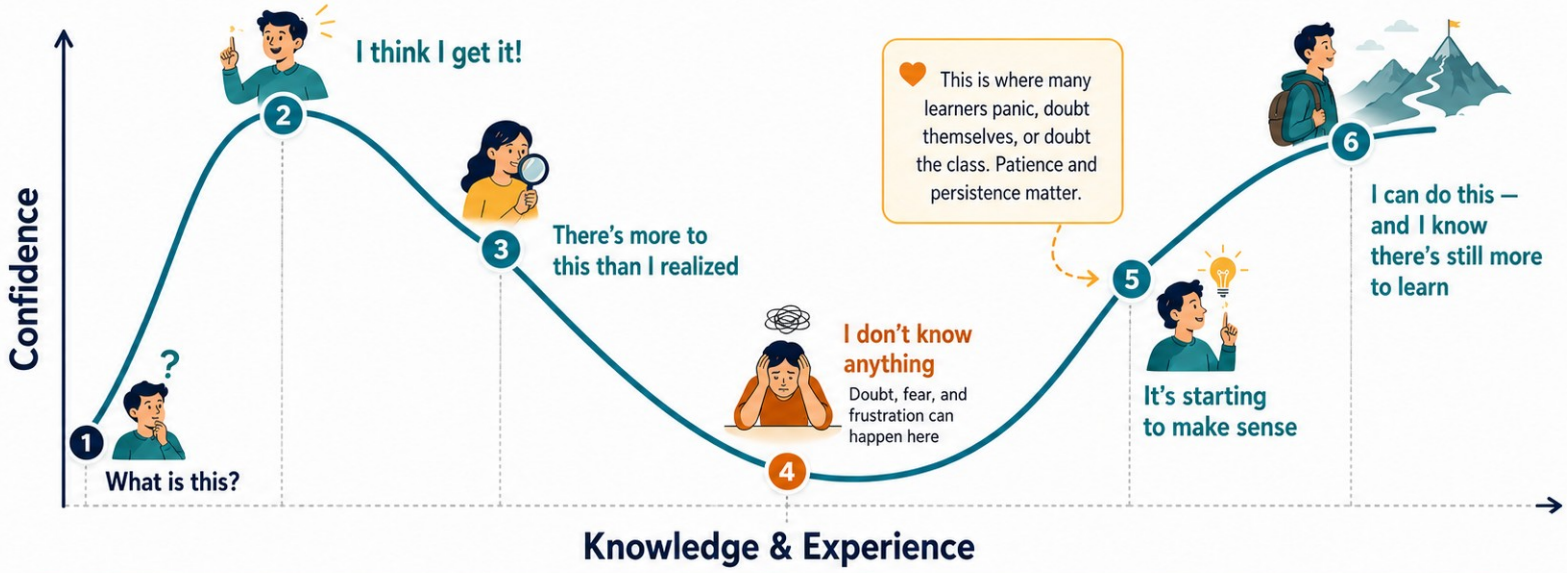
Your 8-week journey - Week 8

Capstone week: Your opportunity to shine, and develop an end-to-end Data Engineering Project covering everything you have learned.



The Learning Curve Is Not Linear

Why learners often feel confident, then overwhelmed, then capable again



Great learning often feels uncomfortable in the middle.
With practice, support, and persistence, confusion turns into competence and humility.

PROGRAM RULES

The AI-Free Zone — Weeks 1 through 4

No GitHub Copilot. No ChatGPT. No LLM-generated code. You write every line by hand.

Why this rule exists:

- **Build syntax muscle memory.** Engineers who rely on AI before they can read code cannot debug, review, or extend it.
- **Develop debugging resilience.** Reading error messages and fixing them yourself is the skill. AI takes it away before you have it.
- **Earn the right to use AI tools.** In Week 6 you get GitHub Copilot — and you will know exactly when to trust it and when not to.

What is data engineering?

“The development, implementation, and maintenance of systems that take raw data and produce high-quality, consistent information for downstream use.”

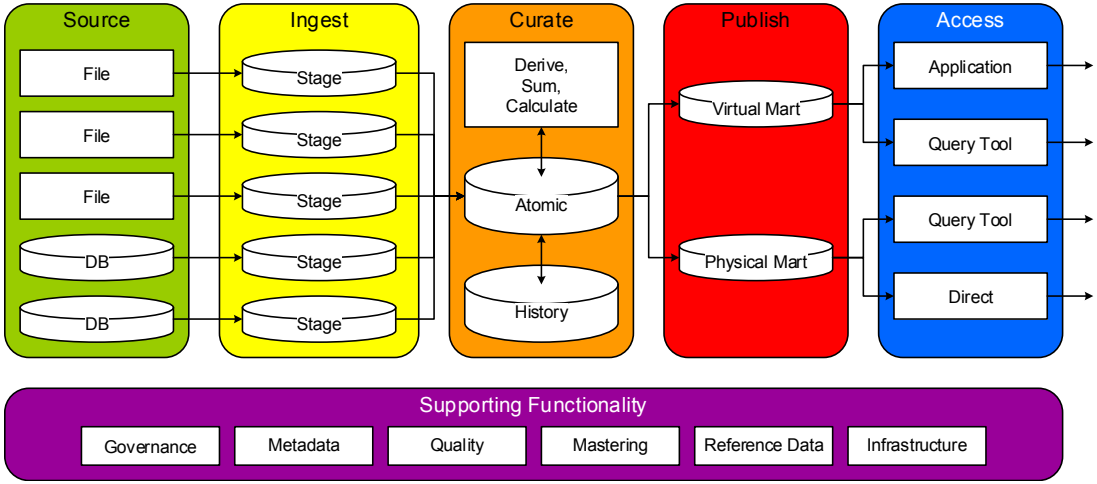
You own everything between the systems that create data and the people who consume it.

Modern data engineering: the full system

Data engineers connect data producers to analytics, AI, and business action — while owning the platform, quality, and reliability in between.

Data Engineering – Once the data structure is sound, data can flow from sources to delivery. Multiple layers are used so that increments of value can be added along a value chain, much like a manufacturing process.

- **Sources** are either our internal operational systems or third-party data sources.
- **Ingest** lands data safely, preserves the truth of the source, and readies it for processing.
- **Curate** unifies the data into a conformed view and applies common rules.
- **Publish** customizes the data to he views of the local organizational area.
- **Access** gets to the data using tools that are best suited to the user's needs.



- Structure is more critical than flow. The prior slide is around structure. This one is around flow.
- Both are important. But of the two, getting the structure right is the bigger win. Prioritize accordingly.

Data engineer vs analyst vs scientist

Data engineer

Builds and operates **pipelines** and platforms

Cares about reliability, scale, cost, quality

SQL, Python, Spark, cloud services, orchestration

Output: trusted, fresh datasets

Data analyst

Answers business questions with data

Cares about metrics, trends, dashboards

SQL, BI tools (Tableau, Looker, ThoughtSpot)

Output: insights and reports

Data scientist

Builds **models** that predict and classify

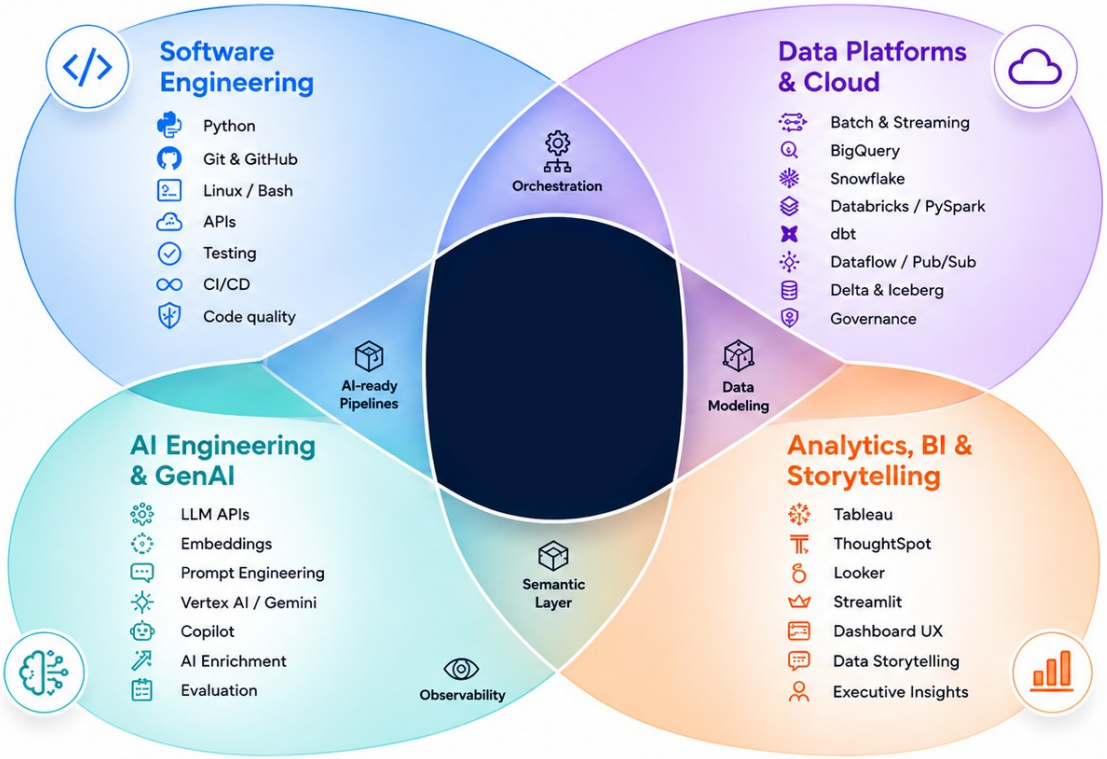
Cares about features, accuracy, experiments

Python, statistics, ML frameworks

Output: models and predictions

Modern Data Engineering

Where software, data platforms, AI, and analytics converge



01

Understanding data

Structured to unstructured, CSV to Parquet, batch to streaming.

Three shapes of data

Structured

Rows and columns, fixed schema

Relational tables, spreadsheets

Insurance: policy records, claims tables, premium ledgers

Semi-structured

Self-describing, flexible schema

JSON, XML, Avro, logs

Insurance: telematics events from cars, API payloads, clickstream

Unstructured

No predefined schema

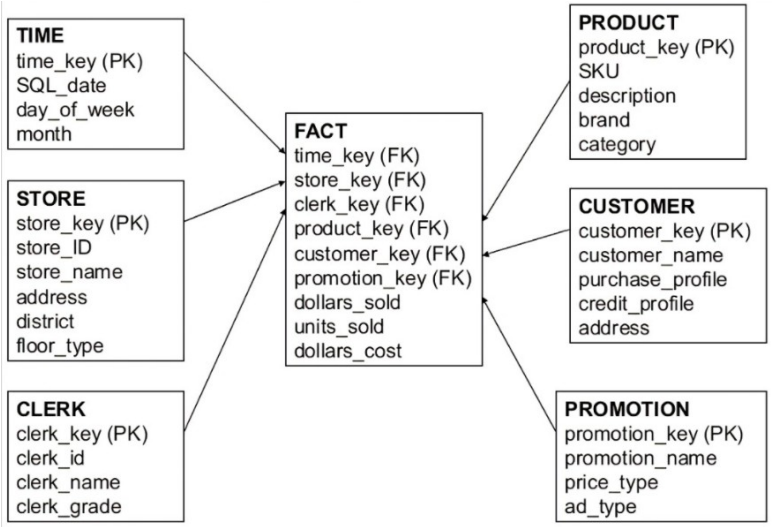
Images, audio, PDFs, free text

Insurance: accident photos, adjuster notes, call transcripts

Most enterprise data is semi- or unstructured and GenAI is making it usable. We return to this in Week 6.

Structured Data – The majority of the data that drives the business is structured. The goal of structuring data is to match the business so it can be used efficiently for insights.

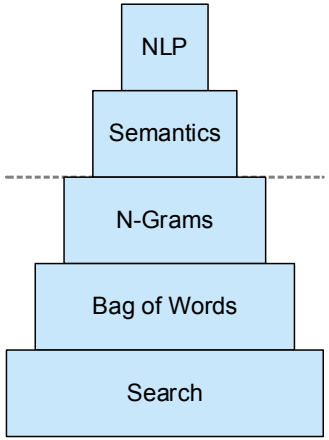
- **Entities.** The things you identify in the real world generally become entities, such as customers and products.
- **Attributes.** Entities have attributes, such as the description or price of a product.
- **Keys.** Entities are identified by keys, which are one or more attributes that uniquely identify the entity.
- **Relationships.** Entities relate to each other based on real-world associations that are captured by carrying an entity's key in another entity.
- **Normalization.** There are eight levels of breaking up or putting together entities.
- **Fit for purpose.** There is no "right answer", just many trade-offs and considerations.



Unstructured Data – An important minority of modern data work involves unstructured information, mainly text. Complexity can be as simple as search and as hard as understanding human conversation.

- 80% of the world's data *content* is unstructured. But 80% of the world's data *value* is structured.
- While a minority of the work, the value is sometimes critical. Core goal is to extract structure.

Text Processing Hierarchy



Natural Language Processing (NLP) – Understanding the nuances of natural language. Think "Watson" on "Jeopardy". Attempts to consider emotion, culture, idioms, humor, plays on words.

Semantics – Understanding meaning. Would know that "claimant declined the officer's repeated offer to call an ambulance" likely means no ambulance was on the scene. Probably knows that "Office Depot" is a business.

N-grams – Considers positions of words relative to each other. "Primary vehicle" is a 2-gram; "unsafe left turn" is a 3-gram; etc. Knows that "office depot" is different from "office" and "depot", but doesn't know it's a business.

Bag of words – Considers all words equal anywhere in a document and assigns no meaning. "Severe" and "injury" 1,000 words apart is the same as right next to each other. Takes out stop words and stems.

Search – Perform text matching. Think search engines circa the 1990's. Matches strings regardless of multiple meanings, context, frequency of the given tokens or other tokens, or variations like "run/ran/running".

Note: This is a useful simplification, not an exhaustive review. Some processing mechanisms use multiple layers. Some layers have multiple sub-layers. Other layers can be added. This view is just useful for overview.

- Learn regular expressions! Every aspect of text processing uses them. (My favorite language!)

File formats every DE must know

Format	Type	Schema	Best for
CSV	Row, text	None (inferred)	Simple exchange, small files, humans
JSON	Nested, text	Self-describing	APIs, events, semi-structured payloads
Parquet	Columnar, binary	Embedded + typed	Analytics at scale — our default in this course
Avro	Row, binary	Embedded + evolvable	Streaming records, schema evolution

Why columnar wins for analytics: queries read only the columns they need, data compresses better, and engines can skip row groups entirely. The NYC Taxi dataset ships as **Parquet** for exactly this reason.

CSV (Comma-Separated Values)

Overview & Usage

What is it? A row-based, plain text format where values are separated by commas. It has no strict embedded schema; types are inferred.

Why & When is it used? Best for simple data exchange, moving small files, and human readability. It is universally supported, making it the default choice for legacy integrations.

Example

```
id,first_name,role,active
1,Alice,Engineer,true
2,Bob,Analyst,false
3,Charlie,Scientist,true
```

JSON (JavaScript Object Notation)

Overview & Usage

What is it? A nested, plain text format that is entirely self-describing, utilizing key-value pairs to represent complex hierarchies.

Why & When is it used? Ideal for web APIs, event-streaming, and semi-structured payloads. It excels when the schema is dynamic and nested records are frequent.

Example

```
{
  "id": 1,
  "first_name": "Alice",
  "skills": [
    "Python",
    "SQL"
  ],
  "active": true
}
```

Apache Parquet

Overview & Usage

What is it? A highly efficient columnar, binary storage format with an embedded and strongly typed schema.

Why & When is it used? The gold standard for analytics at scale. Data compresses better, and analytical engines (like Spark) can skip row groups entirely and only read the specific columns needed for a query.

Example (Conceptual Columnar Layout)

```
id:      [1, 2, 3]
(Stored contiguously, highly compressed)

name:    ["Alice", "Bob", "Charlie"]
(Dictionary encoding applied)

active:  [true, false, true]
(Bit-packing applied)
```

Apache Avro

Overview & Usage

What is it? A row-based, binary format that features an embedded, robust JSON schema within the file.

Why & When is it used? Best for streaming records (e.g., Apache Kafka). Its strict yet highly evolvable schema makes it incredibly reliable for environments where data structures change frequently over time.

Example (Embedded Schema Definition)

```
{
  "type": "record",
  "name": "Employee",
  "fields": [
    {"name": "id", "type": "int"},
    {"name": "name", "type": "string"}
  ]
}
// ... Followed by compact binary data blocks ...
```

File Format vs. Table Format

File Formats

Physical Storage Layer Defines exactly how bits and bytes are organized and compressed on a physical disk (e.g., Parquet, CSV, JSON).

Key Limitations File formats have no concept of a "database table" spanning millions of files. They cannot handle ACID transactions, concurrent writes without corruption, or track historical changes natively.

Table Formats

Logical Metadata Layer A management layer (like Iceberg, Delta) that sits *on top of* physical files (usually Parquet), acting as an intelligent directory.

Key Benefits Abstracts raw files to look like traditional SQL tables. Enables powerful features like ACID transactions, time travel (querying older versions of data), and seamless schema evolution.

Modern Table Formats

Delta Lake

Originally developed by Databricks, deeply integrated with Apache Spark. It provides phenomenal out-of-the-box ACID transactions, unified batch and streaming data processing, and highly optimized time travel capabilities.

Apache Hudi

Standing for "Hadoop Upserts Deletes and Incrementals", Hudi was built primarily for streaming-first architectures. It excels at heavy, continuous row-level updates, upserts, and managing late-arriving data streams.

Apache Iceberg

Created by Netflix, this open standard is highly engine-agnostic (works perfectly with Trino, Flink, Spark). It is unmatched for handling petabyte-scale analytics, utilizing hidden partition evolution and avoiding directory-listing bottlenecks.

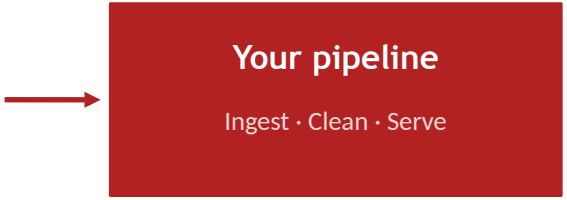
Delta Lake vs Apache Hudi vs Apache Iceberg

The three leading open table formats powering modern data lakes and lakehouses (2026 landscape)

Feature	Delta Lake	Apache Hudi	Apache Iceberg
Primary Strength	Unified batch + streaming, Lakehouse on Databricks	Streaming ingestion, CDC, incremental processing	Large-scale analytics, hidden partitioning, multi-engine
Default Storage	Parquet (+ transaction log in JSON)	Parquet / Avro / ORC + timeline metadata	Parquet / ORC + snapshot/manifest metadata
ACID Transactions	Strong (optimistic concurrency)	Strong (timeline-based)	Strong (snapshot isolation)
Time Travel	Yes — via transaction log history	Yes — via commit timeline & instants	Yes — via snapshots & branches/tags
Schema Evolution	Yes — enforcement + evolution	Yes — with compatibility rules	Yes — very flexible, hidden partitioning
Partitioning	Manual + Liquid Clustering (advanced)	Yes + clustering for performance	Hidden partitioning (most powerful)
Best Ecosystem Fit	Databricks, Spark, Trino, Flink, Snowflake	Spark, Flink, Kafka Streams, Debezium CDC	Spark, Flink, Trino, Presto, Hive, Glue, Snowflake
When to Choose	You are on Databricks or want simplest Lakehouse experience with strong governance	Heavy streaming/CDC workloads or need fine-grained incremental updates	Cloud-agnostic, multi-engine access, or need advanced partitioning & governance at massive scale

Where data comes from

App databases	Policy admin, claims & billing systems (OLTP)
APIs	Telematics API (driving events), rating & weather feeds
Event streams	Mobile app telematics, fraud signals, IoT sensors
Files & feeds	Nightly SFTP claims drops (CSV/EDI), vendor files
SaaS platforms	Guidewire (claims), Salesforce, HR/payroll SaaS



Batch vs streaming

Batch

Bounded chunks on a schedule (hourly, nightly)

Simpler, cheaper, easier to re-run

Latency: minutes to hours

Example: nightly claims summary load

Streaming

Unbounded events processed continuously

Complex, always-on infrastructure

Latency: seconds or less

Example: real-time fraud flags on claims

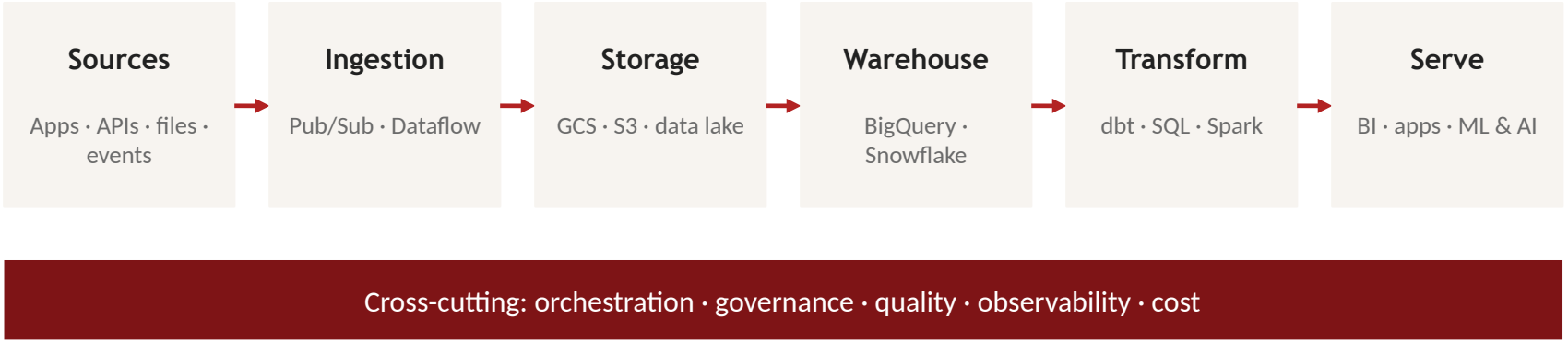
Rule of thumb: start with batch; go streaming only when the business value of low latency justifies the cost. Most pipelines you'll build are batch.

02

The modern data stack

The layers of tooling and where The Hartford's platforms fit.

Anatomy of the modern data stack



Every week of this program maps to one or more of these layers. By Week 8 you will have touched all of them.

The Hartford's data platform

Snowflake

Enterprise analytical warehouse — Week 4 deep dive

Google Cloud

Strategic direction: BigQuery, Dataflow, Vertex AI — Weeks 3–6

AWS

Established footprint: S3, EMR — secondary cloud overlay

BI & analytics

Tableau, Looker, ThoughtSpot — Week 7

GitHub Copilot

AI pair programming — introduced Week 6, after you can code without it

Databricks

Our Spark learning environment (Free Edition — serverless) — Week 5

Tools – Don't be enamored with technology in the DW space. The win in DW work is in the structure and flow. Technology just enables. That said, tools remain important, and we use very good ones.

Tool	Description
Tableau	More flexible data query and reporting by business users.
Dremio	For data federation—getting data from various places, combining it, querying it efficiently.
Dataiku	For data wrangling—cleaning up data in preparation for analysis.
Thoughtspot	Structured-like queries against unstructured data.
InfoSphere	The suite of products used for data governance and quality.
Python	The main programming language of data science and data engineering.
SQL	Universal language of data query. All structured sources speak it. Many unstructured dialects.
Talend	Specialized graphical tool for data extraction, loading, and transformation (ELT).
Elasticsearch	High performance lookups of data. Heavily indexed. Fuzzy logic.
Snowflake	The cloud based database for all data warehousing needs.
Oracle	Major legacy database that we're running off in favor of Snowflake.
S3	Large-scale, low-cost storage platform.
Hadoop/EMR	The big data platform for large-scale work on lots of data.
Autosys	Runs thousands of jobs lights-out every day. Knows dependencies. Tracks status. Sends alerts.
Linux	Foundation on which all other things run. Core utilities. Basic security.
AWS	Amazon Web Services, the cloud provider for The Hartford. Has 400+ services.

Roles in a modern data team

Role	Focus	You'll practice it in
Data engineer	Pipelines, platforms, data quality	Every week
Analytics engineer	SQL transformations, dbt models, semantic layer	Weeks 3–4
Data architect	System design, patterns, trade-offs	Day 5, capstone
Platform / DataOps	CI/CD, orchestration, monitoring	Weeks 4–5
ML / AI engineer	Feature pipelines, model serving, GenAI	Week 6

Titles vary by company — the skills overlap heavily. This program makes you dangerous across all five.

Four types of analytics

Descriptive — what happened?

Monthly claims dashboard; top loss categories last quarter

Diagnostic — why did it happen?

Why did auto claims spike in March? Drill-down and correlation

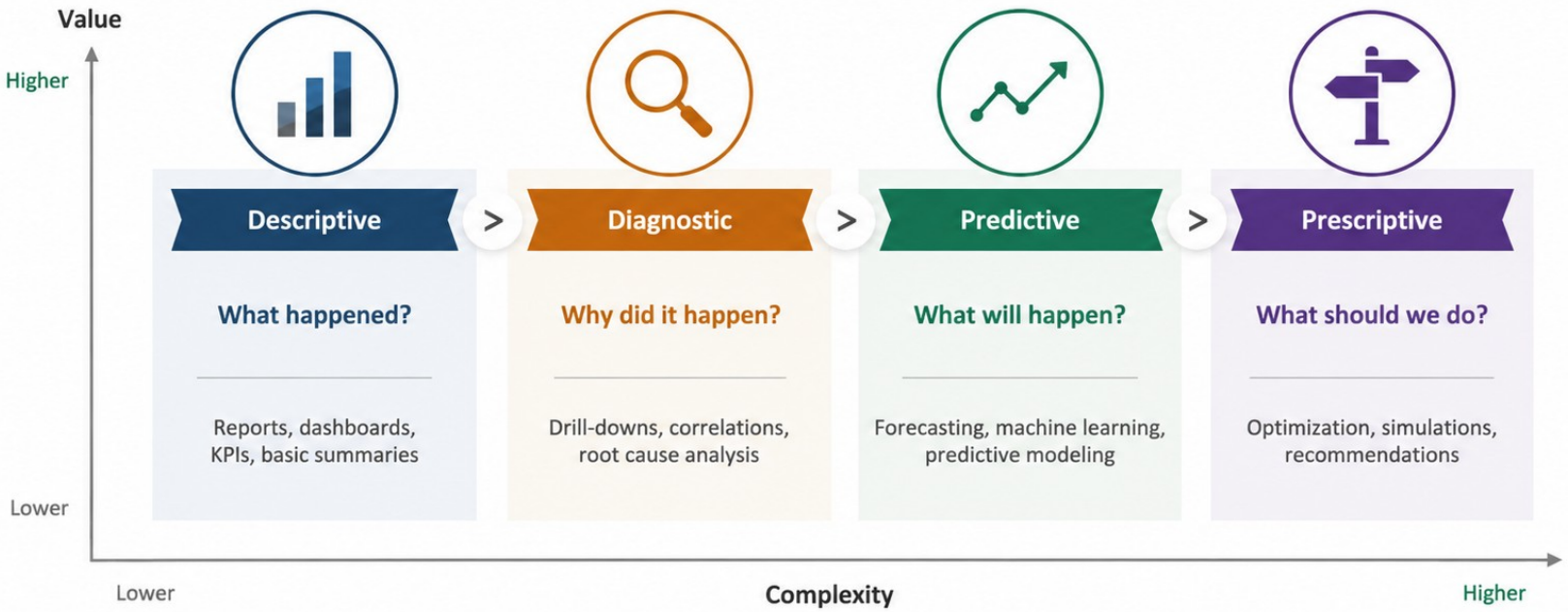
Predictive — what will happen?

Which policies are likely to churn? Expected claim severity

Prescriptive — what should we do?

Recommended premium adjustment; optimal adjuster routing

From insight to impact



03

Your project thread

One pipeline, eight weeks, real data at scale.

PROJECT THREAD

The longitudinal project: NYC Taxi pipeline

W1
Design the architecture

W2
Pull & clean with Python

W3
Load into BigQuery

W4
Model with Snowflake + dbt

W5
Scale with PySpark

W6
Enrich with GenAI

W7
Visualize & serve

W8
Capstone: ship it

Millions of real trip records, in Parquet, with all the messiness of production data.

GROUP ACTIVITY

Activity 1 — Analytics in action

Goal — apply the four analytics types to a real industry and defend your reasoning.

75 min

1. In your group, pick an industry: retail, healthcare, ride-sharing, airline, streaming, or banking.
2. For each analytics type, define one concrete business question and the data needed to answer it.
3. Identify which data shapes (structured / semi / unstructured) each question requires.
4. Build 2–3 slides; every member speaks during the readout.

Deliverable: 5-minute group readout — one analytics question per type, with data requirements.

GROUP ACTIVITY

Activity 2 — DE case study: car insurance

Goal — tell the story of a data engineer's role in a car insurance company.

60 min

1. Identify real examples of structured, semi-structured, and unstructured data in car insurance.
2. Trace one data flow end-to-end: where it's generated, how it's ingested, where it lands, who consumes it.
3. Discuss: what could go wrong? (quality, privacy/PII, latency, scale)
4. Prepare a 5-minute story: “a day in the life of this data.”

Deliverable: 5-minute group story + two slides showing the data flow.

RECAP

Key takeaways

1. Data engineers build the systems that turn raw data into trusted, usable information.

2. Data comes in three shapes — structured, semi-structured, unstructured — and format choice (Parquet!) matters at scale.

3. Batch is the default; streaming is for when seconds matter and the cost is justified.

4. The modern data stack is layered — and The Hartford's stack maps cleanly onto it.

5. Tomorrow: cloud fundamentals — IaaS/PaaS/SaaS, GCP vs AWS, IAM, and billing.